

Parallel Computing Project Report

Matrix Multiplication

Leonardo Bertani

Rafaella Richel Pearl

Academic Year 2025/2026

1 Introduction

Matrix multiplication is one of the most fundamental operations in linear algebra, used for a wide range of applications, including computer graphics, machine learning, and high-performance scientific simulations.

Given the importance of that operation, several hardware manufacturers have developed specialized co-processors and embedded calculators to execute these matrix routines directly at the hardware level.

However, this report focuses on the software-level implementation and optimization of this workload. We first explore a baseline sequential approach, and then we will try to parallelize that algorithm using MPI.

Although the general procedure for matrix multiplication is valid for any pair of matrices where the number of columns in matrix A is equal to the number of rows in matrix B , this report focuses on square matrices of size $N \times N$. This choice is motivated by structural and algebraic symmetry, which simplifies the theoretical analysis and allows for an optimal, balanced mapping of the computational workload onto the parallel processor topology.

2 Sequential algorithm

2.1 Problem definition

Given two square matrices A and B of dimension $N \times N$, the goal is to compute their product $C = A \times B$, where C is also an $N \times N$ matrix.

The element C_{ij} of the resulting matrix is computed by taking the dot product of the i -th row of A and the j -th column of B .
$$C_{ij} = \sum_{k=0}^{N-1} A_{ik} \cdot B_{kj}$$

2.2 Algorithm and complexity

In the following figures, we present the sequential algorithm for matrix multiplication in pseudocode and C code. This follows the approach described in the previous section step-by-step.

Pseudocode

```
Sequential MM(A, B, C, N):  
  Initialize C with all zeros  
  For i = 0 To N - 1:  
    For j = 0 To N - 1:  
      For k = 0 To N - 1:  
        C[i][j] += A[i][k] * B[k][j]
```

C Code

```
void matrix_multiply(double *A,  
                     double *B, double *C, int N) {  
  for (int i = 0; i < N; i++) {  
    for (int j = 0; j < N; j++) {  
      C[i][j] = 0.0;  
      for (int k = 0; k < N; k++) {  
        C[i][j] += A[i][k] * B[k][j];  
      }  
    }  
  }  
}
```

As we can see in the code above, the algorithm is composed of three nested loops. The outer two loops iterate over the rows and columns of the resulting matrix C . The inner loop iterates through the elements of the i -th row of matrix A and the j -th column of matrix B simultaneously to compute their dot product, following the standard row-by-column multiplication strategy.

About the time complexity, this algorithm has a time complexity of $\Theta(N^3)$. Then, regarding the space complexity, the algorithm requires $\Theta(N^2)$ auxiliary space to store the input matrices A , B , and the resulting matrix C in memory.

3 Parallel design

Some considerations must be addressed before trying to parallelize the algorithm. In particular, we can analyze the dependency of the algorithm and the communication pattern, to ensure an optimal parallel implementation.

3.1 Dependency analysis

Before parallelizing the algorithm, it is critical to analyze how data flows through the hardware. While mathematically straightforward, the sequential i - j - k loop order introduces a significant performance bottleneck due to its asymmetric memory access pattern:

- Matrix A is accessed row-by-row ($A[i][k]$), which perfectly aligns with the row-major storage of C arrays. This benefits from excellent *spatial locality* in the cache.
- Matrix B is traversed column-by-column ($B[k][j]$). This creates a stride- N access pattern, loading an entire cache line but only using a single element before moving to the next row. As N grows, this triggers a massive rate of *cache misses*.

A dependency analysis reveals a high degree of data parallelism when mapping this workload to multiple execution units. The iterations of the two outer loops (i and j) are completely independent, since computing any element C_{ij} does not depend on, nor update, any other element of the output matrix C .

From a practical implementation standpoint, this independent structure allows for a straightforward *1D Domain Decomposition* (slicing a one-dimensional computational domain into smaller, consecutive segments). The algorithm can be parallelized by assigning a contiguous subset of rows of matrix A to each thread or process. Each execution unit can then compute its corresponding rows of matrix C independently.

3.2 Communication pattern

While the 1D decomposition eliminates the need for synchronization in a shared-memory system (e.g., OpenMP) where all threads access a single global RAM, it introduces severe scalability limits in a distributed-memory system (e.g., MPI). In a distributed environment, a 1D approach requires a communication pattern where every single process must replicate and hold the entire matrix B in its private memory, saturating network bandwidth via heavy broadcast operations.

To overcome this network bottleneck, the workload can be restructured into a *2D Domain Decomposition*, creating a logical 2D grid of processes. Under this scheme, each process is responsible for computing only a localized submatrix (block) of C , requiring access to just a submatrix of A and a submatrix of B .

However, in a naive 2D parallel implementation, a processor still needs to perform collective communication (broadcasts) across its entire virtual row and column to fetch the required blocks of A and B over time. This communication overhead heavily bounds scalability, driving the need for a fully localized, communication-minimizing topology configuration, such as Cannon’s Algorithm.

4 Topology-aware implementation

4.1 Topologies considered

In a matrix multiplication algorithm, the computation naturally operates on a fixed geometric space defined by its nested loops. When mapping these loop structures to parallel hardware, closed topologies like the 1D Ring and 2D Torus offer significant code-level and algorithmic advantages over open topologies like the Array or Mesh.

For instance, closed topologies are perfectly symmetric, meaning every node has the exact same number of neighbors. This structural uniformity allows every processing unit to execute the identical chunk of code without requiring complex conditional statements to handle boundary exceptions. Additionally, physically bridging the boundaries cuts the maximum routing length in half, allowing the system to optimize the direction in which data is sent and drastically reducing latency during the data-shifting phases.

4.2 Embedding the communication pattern

When embedding the logical communication pattern of the matrix multiplication loops onto the hardware, the choice between a 1D Ring and a 2D Torus fundamentally alters how data flows through the network and how the code is structured.

4.2.1 1D Ring Embedding

In a 1D Ring configuration, the multi-dimensional problem must be compressed into a single linear dimension. The matrix is typically divided into horizontal slices (row blocks), and each node communicates only with its two linear neighbors.

While this topology naturally supports the sequential rotation of one matrix axis, it creates a severe structural bottleneck for the remaining dimensions. Because the hardware lacks a second physical dimension, any data reuse or shifting required by the nested loops along the column axis cannot be localized. Data must either be globally replicated across the entire ring or bounced across multiple intermediate nodes, scaling the communication latency linearly with the number of processors.

4.2.2 2D Torus Embedding

The 2D Torus provides a perfect geometric match for the nested loop structure of matrix multiplication, as it maps the execution directly onto a two-dimensional physical grid of processors.

By having four physical connections per node with wrap-around links (so the degree is 4 for every node, also on the boundaries), the communication pattern achieves optimal spatial locality in both dimensions simultaneously. The cyclic data shifts required by the outer and inner loops are executed through independent, concurrent paths. Data moves exactly one hop to an immediate neighbor in a single step, preventing network contention. This multi-directional routing eliminates the non-local data traffic inherent to the 1D Ring, keeping the communication overhead constant and ensuring maximum computational efficiency as the processor grid scales.

5 MPI implementation

5.1 Language and build

For this project, the C programming language and the MPI library were selected to implement the parallel matrix multiplication algorithm.

Initially, Python was considered as an alternative. However, standard Python is inherently slow for low-level memory operations, while high-level numerical libraries like NumPy abstract away and automatically optimize matrix computations under the hood. Since NumPy handles memory allocation and parallelism internally, it prevents the explicit control and manual implementation of low-level data movement and communication patterns required by this project. Therefore, C was chosen to ensure maximum execution speed, precise memory management, and total control over the MPI communication primitives.

5.2 Topology realisation in MPI

To implement a topology-aware algorithm like Cannon's, the abstract concept of a 2D Torus must be explicitly instantiated within the MPI execution environment.

By default, MPI initializes processes in a flat, unstructured communicator (`MPI_COMM_WORLD`),

which provides no notion of geometric proximity or neighborhood. To expose this structure to the runtime, three MPI primitives are composed in sequence:

1. **Grid Creation and Periodicity** (`MPI_Cart_create`): restructures the flat process list into a logical 2D grid of size $\sqrt{P} \times \sqrt{P}$. Setting the periodicity flags to `{1, 1}` closes both dimensions into a torus, ensuring that every process has exactly four neighbors with no boundary exceptions.
2. **Neighbor Discovery** (`MPI_Cart_shift`): resolves the MPI ranks of the four cardinal neighbors (left, right, up, down) for each process. These are computed once prior to the main loop and reused across both the skewing phase and the iterative Cannon shifts, avoiding repeated rank lookups at runtime.
3. **Deadlock-Free Data Exchange** (`MPI_Sendrecv`): encapsulates the outgoing send and incoming receive into a single atomic operation. This is critical in a ring-shift pattern where every process simultaneously sends and receives — using separate `MPI_Send` and `MPI_Recv` calls would cause all processes to block indefinitely waiting for a listener.

5.2.1 Initial skewing

Before the main calculation starts, Cannon’s algorithm uses a setup step to align the data: row i of matrix A shifts left by i positions, and column j of matrix B shifts up by j positions. At first, we tried to move this data all at once using a single `MPI_Cart_shift` command. This worked for small processor grids ($P \leq 9$), but caused incorrect math results on larger grids ($P \geq 16$). The issue was that sending the data in one big jump skipped over the processors in between, which broke the 2D network layout. We fixed this by moving the data one step at a time to direct neighbors. This passed the blocks correctly through the network and produced the exact same results as the standard sequential version.

6 Experimental setup

To evaluate the correctness, speedup, and overall scalability of the MPI implementation of Cannon’s Algorithm, benchmarks were conducted across two distinct hardware environments. This dual-setup approach allowed for local validation of smaller processor grids before deploying the algorithm for massive parallelization.

6.1 Local Computing Environment

Initial development were established on a local Linux Ubuntu workstation (Intel Core i7 12th Gen, 32 GB RAM). This environment was also utilized to validate the parallel algorithm on smaller grids ($P = 4$ and $P = 9$).

6.2 High-Performance Computing Environment (CAPRI)

Large-scale benchmarking was executed on the University of Padua’s CAPRI HPC infrastructure. CAPRI features a Cache-Coherent NUMA architecture with 16 CPUs (256 physical cores) and 6 TB of globally shared RAM, connected via an HPE proprietary Intel UPI central

hub. Tests were compiled with OpenMPI and managed via SLURM, evaluating grids up to 8x8 (64 processors) and matrix sizes up to 5000x5000.

7 Results

7.1 Execution Time and Speedup

Table 1 shows the local machine results for both process counts side by side, and Table 2 combines all three CAPRI configurations ($P = 4$, $P = 16$, $P = 25$).

For $P = 9$, N must be divisible by 3; for $P = 25$, divisible by 5. This is why these configurations use different N values, and why neither sub-table in Table 1 shares rows. For $P = 25$, the $N = 4$ case failed at runtime and is excluded (—).

Table 1: Local machine results: $P = 4$ (2×2) and $P = 9$ (3×3)

$P = 4$				$P = 9$			
N	Seq (s)	Par (s)	Speedup	N	Seq (s)	Par (s)	Speedup
4	0.000001	0.000251	0.00	255	0.056660	0.013643	4.15
200	0.016312	0.007559	2.16	510	1.096061	0.578107	1.90
500	0.327045	0.116566	2.81	765	7.727348	1.833078	4.22
800	1.530781	0.562793	2.72	1020	11.789010	4.048152	2.91
1000	3.486282	1.121739	3.11	2040	35.915316	7.229086	4.97
2000	32.714191	10.522947	3.11	3072	349.357671	90.501574	3.86
4000	394.880910	126.129741	3.13	4095	676.634829	94.353462	7.17

Table 2: CAPRI results: $P = 4$ (2×2) and $P = 16$ (4×4)

N	Seq (s)	$P = 4$			$P = 16$		
		Par (s)	Speedup	Compute%	Par (s)	Speedup	Compute%
4	0.000001	0.000608	0.00	0.07%	0.169457	0.00	0.00%
200	~0.042	0.207911	0.20	4.62%	0.427988	0.10	0.55%
500	~0.867	0.173822	4.88	96.02%	0.475445	1.83	8.03%
800	~3.323	0.857277	3.78	98.73%	0.699960	5.00	23.49%
1000	~6.206	1.756798	3.37	98.86%	1.079994	6.28	31.34%
2000	~67.6	17.375455	4.28	99.67%	9.490967	7.33	38.47%
4000	~1036	141.643071	5.86	98.11%	62.854408	13.36	52.84%

7.2 Speedup and Efficiency

For small matrices ($N \leq 200$), the parallel implementation underperformed the sequential baseline across all topologies due to MPI initialization and communication overhead. As computational volume scaled, the parallel efficiency rapidly improved. At $N = 4000$, the $P = 25$ configuration achieved a remarkable 34.4x speedup (44.66s vs 1036s). This superlinear speedup is heavily attributed to cache effects: distributing the workload across 25 nodes reduced the block sizes to easily fit within the L1/L2 caches of the individual processors, eliminating the cache-miss bottleneck observed in the sequential baseline.

Table 3: CAPRI results: $P = 25$ (5×5)

N	Seq (s)	Par (s)	Speedup	Compute%
4	—	—	—	—
200	0.042013	0.439988	0.10	0.35%
500	0.883308	0.568006	1.56	5.20%
800	3.221853	0.660117	4.88	16.30%
1000	5.914401	0.592395	9.98	34.67%
2000	59.039703	4.950587	11.93	41.39%
4000	1538.387289	44.661665	34.45	47.07%

7.3 Computation vs. Communication

The data highlights a strict tradeoff. At $P = 4$, computation dominates ($>98\%$ time for $N > 500$). However, for larger grids like $P = 25$, communication overhead accounts for over 50% of the total time until N scales massively, proving that adding processors requires exponentially larger datasets to remain efficient.

8 Discussion

8.1 Limitations

This algorithm strictly requires that the global matrices A and B are square ($N \times N$). Furthermore, for the parallel implementation of Cannon’s algorithm, the total number of MPI processes (P) must be a perfect square, meaning that the logical grid size $q = \sqrt{P}$ must be an integer.

A major practical constraint is that the global matrix dimension N must be perfectly divisible by q ($N \pmod{q} = 0$).

If these structural conditions are not met, the workload cannot be evenly distributed across the 2D torus topology without implementing complex padding or masking techniques, which would significantly degrade parallel efficiency.

Additionally, as a standard 2D matrix multiplication method, Cannon’s algorithm is bound to a strict communication lower bound of $\Omega(N^2/\sqrt{P})$ words sent or received per processor.

8.2 Possible improvements

We could explore alternative parallelization strategies to reach a more flexible solution in handling non-square matrices or processor grids that are not perfectly square.

Furthermore, communication overhead can be mitigated by upgrading from a 2D topology to a 3D parallel matrix multiplication algorithm. While 2D algorithms like Cannon’s are strictly bound by a communication lower bound of $\Omega(N^2/\sqrt{P})$ words sent or received per processor, 3D approaches replicate matrix data across a three-dimensional processor grid. This technique reduces the total network communication volume per processor to $\Omega(N^2/P^{2/3})$, though it does so at the expense of an increased local memory footprint. [1]

However, the asymptotic reduction in communication achieved by a 3D layout compared to a 2D algorithm like Cannon is limited to a factor of only $P^{1/6}$, meaning that the overhead of data replication may not always be profitable for standard cluster sizes. [1]

To bridge this gap, a 2.5D variant can be introduced [2]. This method incorporates a tunable replication factor that dynamically scales according to the available RAM to reach optimal, communication-avoiding bounds. Additionally, adapting these advanced mapping strategies to higher-dimensional physical interconnects, such as hypercube network topologies [3], can further minimize routing latency and maximize bandwidth, yielding massive speedups on thousands of processing cores.

9 Conclusions

This project implemented and validated Cannon’s algorithm for parallel matrix multiplication on a 2D Torus topology using MPI, confirming both its correctness and its ability to deliver significant speedups over a sequential baseline on large matrices. The results show a clear pattern: for small matrices, communication overhead dominates and parallelism makes things slower, while for large matrices computation takes over and more processes consistently wins. The 2D Torus proved to be a natural fit for the algorithm’s communication pattern, and the gap in speedup between smaller and larger process grids at large N suggests that scaling further would continue to be worthwhile.

10 Acknowledgments

The computational results presented in this report were achieved using the CAPRI computing infrastructure. We gratefully acknowledge the resources provided by the University of Padova Strategic Research Infrastructure Grant 2017: “CAPRI: Calcolo ad Alte Prestazioni per la Ricerca e l’Innovazione”.

References

- [1] D. Irony, S. Toledo, and A. Tiskin, “Communication lower bounds for distributed-memory matrix multiplication,” *Journal of Parallel and Distributed Computing*, vol. 64, no. 9, pp. 1017–1026, 2004, doi: [10.1016/j.jpdc.2004.03.021](https://doi.org/10.1016/j.jpdc.2004.03.021).
- [2] E. Solomonik and J. Demmel, “Communication-optimal parallel 2.5D matrix multiplication and LU factorization algorithms,” in *Euro-par 2011 parallel processing*, in Lecture notes in computer science, vol. 6853. Berlin, Heidelberg: Springer, 2011, pp. 90–109. doi: [10.1007/978-3-642-23397-5_10](https://doi.org/10.1007/978-3-642-23397-5_10).
- [3] J. Berntsen, “Communication efficient matrix multiplication on hypercubes,” *Parallel Computing*, vol. 12, no. 3, pp. 335–342, 1989, doi: [10.1016/0167-8191\(89\)90031-8](https://doi.org/10.1016/0167-8191(89)90031-8).
- [4] Wikipedia contributors, “Matrix multiplication.” https://en.wikipedia.org/wiki/Matrix_multiplication, 2026.